</>

MCS-220 · SECTION 2 · WEB TECHNOLOGIES LAB

Lab Handbook

Installation Guide &

Step-by-Step Program Execution

JDK · JAVA_HOME · Apache Tomcat · NetBeans · Eclipse

Servlets · web.xml · Form handling · GET vs POST

JSP directives, expressions & implicit objects

Spring Boot MVC · Thymeleaf · Bootstrap · WebJars

Syntax explained line by line · Troubleshooting · Lab sessions 1–10

Companion notes for the 2-hour teaching session. Every program in this handbook was compiled and run before publishing.

Contents

1. How to use this handbook 4

1.1 Versions used and tested 4

2. Installation guide 5

2.1 Minimum system requirements 5

2.2 Step 1: Install the Java Development Kit (JDK) 5

2.3 Step 2: Set JAVA_HOME and PATH 5

2.4 Step 3: Install Apache Tomcat 9 6

2.5 Step 4: Install Apache NetBeans and register Tomcat 7

2.6 Step 5: Install Eclipse IDE and Spring Tools 7

2.7 Optional: MySQL (needed from Session 1 exercise 5 and Sessions 6–10) 7

2.8 Verification checklist 8

3. Program 1: Display request information with a Servlet 9

3.1 Goal 9

3.2 Create the project in NetBeans 9

3.3 Source code: RequestServlet.java 10

3.4 Syntax explained 11

3.5 Deployment descriptor: WEB-INF/web.xml 11

3.6 Run the program 13

3.7 Expected output 13

3.8 Try this 13

4. Program 2: Capture a form with two Servlets 14

4.1 Goal 14

4.2 Create the servlets 14

4.3 Source code: PersonalInfoFrom.java (shows the form) 15

4.4 Source code: PersonalInfo.java (handles the submission) 16

4.5 Syntax explained 17

4.6 Run the program 17

4.7 Expected output 17

4.8 Try this: GET vs POST 17

5. Program 3: The same form with JSP 19

5.1 Goal 19

5.2 How a JSP runs 19

5.3 Create the pages 19

5.4 Source code: stdForm.jsp 20

5.5 Source code: studentView.jsp . . . 21
5.6 Syntax explained . . . 21
5.7 Run the program . . . 22
5.8 Try this . . . 22
6. Program 4: Spring Boot MVC form with Thymeleaf and Bootstrap . . . 23
6.1 Goal . . . 23
6.2 Generate the project with Spring Initializr . . . 23
6.3 Open the project in Eclipse . . . 23
6.4 Final project structure . . . 24
6.5 pom.xml . . . 24
6.6 application.properties . . . 27
6.7 The Model: entity/Student.java . . . 28
6.8 The Controller: controller/StudentController.java . . . 29
6.9 The Views: templates/stdForm.html and templates/dashboard.html . . . 31
6.10 Run the application . . . 33
6.11 Expected output . . . 34
6.12 Try this . . . 34
7. Adding Bootstrap, jQuery and Font Awesome . . . 35
7.1 Method 1: WebJars through Maven (works offline) . . . 35
7.2 Method 2: CDN links (needs internet when the page loads) . . . 35
7.3 Bootstrap classes used in the lab . . . 36
8. Troubleshooting . . . 37
9. Lab assignments (session-wise) . . . 38
10. Quick reference . . . 41
10.1 Servlet API . . . 41
10.2 Spring MVC annotations . . . 41
10.3 Thymeleaf . . . 41
10.4 Useful HTTP status codes . . . 42
10.5 Official links . . . 42

1. How to use this handbook

This handbook accompanies the two-hour Web Technologies Lab session (MCS-220, Section 2). The slides explain *why*; this handbook gives you *every step* to install the tools and run each program on your own computer, with the syntax explained line by line.

- **Chapter 2** installs and verifies the full toolchain (JDK, Tomcat, NetBeans, Eclipse).
- **Chapters 3 to 6** are four complete programs. Each one follows the same pattern: **Goal** → **Create** → **Code** → **Syntax explained** → **Run** → **Expected output** → **Try this**.
- **Chapter 7** shows the two ways to add Bootstrap, jQuery and Font Awesome.
- **Chapters 8 to 10** hold troubleshooting, the ten lab-session exercise lists, and a one-page quick reference.

1.1 Versions used and tested

Component	Version used for testing	Notes
JDK	17 or 21 (LTS)	Spring Boot 3 and later need Java 17+.
Servlet/JSP programs	javax.servlet API 4.0 on a Servlet 4 container	Run on Apache Tomcat 9.0.x exactly as written.
Spring Boot program	Spring Boot 4.1.2 (start.spring.io default)	Uses jakarta.* imports; Java 17.
Bootstrap / jQuery / Font Awesome	5.3.3 / 3.7.1 / 6.5.2 (WebJars)	Any recent version works.

IMPORTANT **javax vs jakarta.** The lab manual's servlet code imports `javax.servlet.*`. Tomcat 10 and later renamed this package to `jakarta.servlet.*`. So: use **Tomcat 9** with the code exactly as printed here, or, on Tomcat 10/11, replace every `javax.servlet` with `jakarta.servlet`. Mixing them causes HTTP 500 / ClassNotFound errors.

NOTE The manual was written for Spring Boot 2.5 and Java 11. Spring Initializr no longer offers those versions, so Chapter 6 uses the current release. The code is the same except for two details that are pointed out where they occur (jakarta imports and the WebJars locator artifact).

2. Installation guide

2.1 Minimum system requirements

Item	Requirement
Operating system	Windows 10/11 (64-bit), Ubuntu 20.04+ or macOS 12+
Memory / disk	8 GB RAM recommended (4 GB minimum); about 3 GB free disk
Internet	Needed for downloads, the Eclipse installer and Maven dependency downloads
Admin rights	Needed to install software and edit system environment variables

2.2 Step 1: Install the Java Development Kit (JDK)

Check whether Java is already installed

Open **Command Prompt** (Windows) or **Terminal** (Linux/macOS) and type:

```
Command
java --version
javac --version
```

If both commands print a version of 17 or higher, skip to section 2.3. If you see *'java' is not recognized*, install the JDK.

Windows

1. Go to **oracle.com/java/technologies/downloads** (or **adoptium.net** for the free Eclipse Temurin build).
2. Choose an **LTS** version (17 or 21) and download the **Windows x64 Installer (.exe or .msi)**.
3. Run the installer as administrator and accept the defaults. Note the install folder, for example `C:\Program Files\Java\jdk-17`.
4. Continue with section 2.3 to set JAVA_HOME and PATH.

Linux (Ubuntu / Debian)

```
Terminal
sudo apt update
sudo apt install openjdk-17-jdk
java -version
```

macOS

Download the macOS **.pkg** installer (Arm64 for Apple-silicon Macs, x64 for Intel Macs) and double-click it. Or, with Homebrew: `brew install openjdk@17`.

2.3 Step 2: Set JAVA_HOME and PATH

JAVA_HOME tells tools such as Tomcat, Maven and the IDEs where Java is installed. **PATH** tells the operating system where to find the `java` and `javac` commands.

Windows 10 / 11

1. Press **Start**, type **environment** and open **Edit the system environment variables**.
2. In the **System Properties** window click **Environment Variables…**.
3. Under **System variables** click **New…**. Variable name: `JAVA_HOME`; Variable value: your JDK folder, e.g. `C:\Program Files\Java\jdk-17` (use **Browse Directory…** to avoid typing mistakes). Click OK.
4. Select the **Path** system variable → **Edit…** → **New** → type `%JAVA_HOME%\bin` → move it to the top → OK.
5. Click **OK** in every open window to save.
6. **Close and reopen** Command Prompt, then run `java --version` and `echo %JAVA_HOME%`.

Linux

Edit `/etc/profile` (all users, needs sudo) or `~/.bashrc` (only you) and add at the end:

/etc/profile or ~/.bashrc

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export PATH=$PATH:$JAVA_HOME/bin
# optional: extra jars for command-line compiling
export CLASSPATH=$CLASSPATH:.
```

Activate and verify:

Terminal

```
source /etc/profile        # or: source ~/.bashrc
echo $JAVA_HOME
java -version
```

Expected output

```
openjdk version "17.0.x" ...
OpenJDK Runtime Environment ...
OpenJDK 64-Bit Server VM ...
```

TIP Find the exact JDK folder on Linux with `readlink -f $(which javac)` and remove the trailing `/bin/javac`.

2.4 Step 3: Install Apache Tomcat 9

Tomcat is the web server (servlet container) that runs Servlets and JSP pages.

1. Open **tomcat.apache.org** → **Download** → **Tomcat 9**.
2. Under *Binary Distributions* → *Core* download the **64-bit Windows zip** (Windows) or the **tar.gz** (Linux/macOS).
3. Extract it to a short path without spaces, for example `D:\Apache\apache-tomcat-9.0.x` or `~/apache-tomcat-9.0.x`.
4. Do not start it by hand yet: NetBeans will start and stop it for you (section 2.5).

To test Tomcat on its own: run `bin\startup.bat` (Windows) or `bin/startup.sh` (Linux/macOS), open **http://localhost:8080** and you should see the Tomcat welcome page. Stop it with `shutdown.bat` / `shutdown.sh`.

IMPORTANT The manual's screenshots show Tomcat 10.0.8. If you prefer Tomcat 10 or 11, that is fine, but change the servlet imports to `jakarta.servlet` (see 1.1).

2.5 Step 4: Install Apache NetBeans and register Tomcat

1. Download the installer for your OS from **netbeans.apache.org/download**.
2. Right-click the installer → **Run as administrator** → Yes. A progress bar shows *Configuring the installer…*.
3. On the welcome screen click **Next** (use **Customize…** only if you want to change packs).
4. Tick **I accept the terms in the license agreement** → Next.
5. Choose the installation folder (1st Browse button) and the JDK (2nd Browse button) → Next.
6. Review the summary → **Install** → wait → **Finish**.
7. Launch NetBeans from the Start menu or desktop icon and close the Start Page.

Register Tomcat in NetBeans (one time)

1. **Tools** → **Servers** → **Add Server…** (or use **Add…** while creating a project).
2. Select **Apache Tomcat or TomEE** → Next.
3. **Server Location**: Browse to the Tomcat folder from 2.4.
4. Enter a manager **Username** and **Password** (e.g. admin / admin123) and tick **Create user if it does not exist** → **Finish**.

> **NOTE** The first time you create a web project NetBeans may show **Finding Feature**. Click **Download and Activate** to enable Java Web/EE support, then continue.

2.6 Step 5: Install Eclipse IDE and Spring Tools

1. Download the Eclipse Installer from **eclipse.org/downloads** (`eclipse-inst-jre-win64.exe` on Windows).
2. Run it as administrator and choose **Eclipse IDE for Enterprise Java and Web Developers**.
3. Confirm the **Java VM** (your JDK) and the **Installation Folder**; keep *create start menu entry* and *desktop shortcut* ticked → **INSTALL**.
4. Click **Accept Now** on the Eclipse Foundation Software User Agreement. Installation downloads files, so stay online.
5. Click **LAUNCH**, choose a **workspace** folder (where your projects are stored) → Launch.
6. Install Spring support: **Help** → **Eclipse Marketplace…** → search **Spring** → **Spring Tools** (latest) → Install → Confirm → accept the licence → Finish → **Restart** Eclipse.

IntelliJ IDEA (Community or Ultimate) also opens Maven/Spring projects directly with **File** → **Open**.

2.7 Optional: MySQL (needed from Session 1 exercise 5 and Sessions 6–10)

1. Install **MySQL Community Server 8.x** and **MySQL Workbench** from dev.mysql.com (Windows: MySQL Installer).
2. Set a root password and remember it; it goes into `application.properties` later.
3. Create the lab database: `CREATE DATABASE Student_Admission;`

2.8 Verification checklist

Check	Command / action	Expected result
Java runtime	java --version	17 or higher
Java compiler	javac --version	Same version as java
JAVA_HOME	echo %JAVA_HOME% / echo $JAVA_HOME	Your JDK folder
Tomcat	startup.bat → http://localhost:8080	Tomcat welcome page
NetBeans	Tools → Servers	Apache Tomcat listed
Eclipse	Help → About	Spring Tools listed in installed features

3. Program 1: Display request information with a Servlet

3.1 Goal

Write a servlet that prints the current date and time and details of the incoming HTTP request (method, URI, protocol, path info and client IP address). This is Example 1 of the lab manual and covers Session 1 exercises 1 and 3.

3.2 Create the project in NetBeans

1. **File** → **New Project**.
2. Categories: **Java with Ant** → **Java Web**; Projects: **Web Application** → Next.
3. Project Name: `ServletLab`; keep the default location → Next.
4. Server: **Apache Tomcat** (registered in 2.5); Java EE version: **Java EE 8 Web** (or Java EE 7 Web); Context Path: `/ServletLab` → Next.
5. Frameworks: leave everything unticked → **Finish**.
6. Right-click **Source Packages** → **New** → **Servlet…**. Class Name: `RequestServlet` → Next.
7. Tick **Add information to deployment descriptor (web.xml)**. URL Pattern: `/RequestServlet` → **Finish**.

NetBeans now generates a servlet skeleton and adds it to **WEB-INF/web.xml**. Replace the generated class with the code below.

Project structure (NetBeans *Projects* view):

Directory layout

```
ServletLab
├── Web Pages
│   ├── WEB-INF
│   │   └── web.xml            <- deployment descriptor
│   ├── index.html
│   ├── stdForm.jsp             <- Program 3
│   └── studentView.jsp         <- Program 3
├── Source Packages
│   └── <default package>
│       ├── RequestServlet.java    <- Program 1
│       ├── PersonalInfoFrom.java  <- Program 2
│       └── PersonalInfo.java      <- Program 2
└── Libraries  (JDK + Apache Tomcat)
```

3.3 Source code: RequestServlet.java

Source Packages/RequestServlet.java

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

public class RequestServlet extends HttpServlet {

    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        try (PrintWriter out = response.getWriter()) {
            java.util.Date date = new java.util.Date();
            out.println("Current Date & Time: " + date.toString());
            out.println("<h3>Request Information Example</h3>");
            out.println("Method: " + request.getMethod() + "<br>");
            out.println("Request URI: " + request.getRequestURI() + "<br>");
            out.println("Protocol: " + request.getProtocol() + "<br>");
            out.println("PathInfo: " + request.getPathInfo() + "<br>");
            out.println("Remote Address: " + request.getRemoteAddr());
        }
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }

    @Override
    public String getServletInfo() {
        return "Displays request information";
    }
}
```

3.4 Syntax explained

Code / syntax	What it means
`import javax.servlet.*`	Brings in the Servlet API classes. They are supplied by Tomcat, so nothing extra needs downloading. (Tomcat 10+: jakarta.servlet.*)
`public class RequestServlet extends HttpServlet`	Our class inherits from HttpServlet, which is what makes it a servlet that Tomcat can create and call.
`processRequest(request, response)`	A helper method NetBeans generates. Both doGet() and doPost() call it so GET and POST behave the same.
`throws ServletException, IOException`	Declares the two checked exceptions a servlet may raise; Tomcat handles them and shows an error page.
`response.setContentType("text/html;charset=UTF-8")`	Sets the Content-Type header: the browser will render the body as HTML using UTF-8 characters. Call it before getWriter().
`try (PrintWriter out = response.getWriter())`	try-with-resources: gets a writer connected to the response body and closes it automatically at the end of the block.
`out.println("...")`	Writes text into the response. HTML tags such as <h3> and are interpreted by the browser.
`new java.util.Date()`	Creates an object holding the current date and time; toString() formats it, e.g. Mon Nov 09 16:07:34 IST 2026.
`request.getMethod()`	HTTP method used: GET (typing a URL / link) or POST (a form with method="post").
`request.getRequestURI()`	The path part of the URL: context path + servlet path, e.g. /ServletLab/RequestServlet.
`request.getProtocol()`	Protocol and version, normally HTTP/1.1.
`request.getPathInfo()`	Extra path after the servlet mapping. null for an exact mapping such as /RequestServlet.
`request.getRemoteAddr()`	IP address of the client. 127.0.0.1 or 0:0:0:0:0:0:0:1 when the browser runs on the same machine.
`@Override`	Tells the compiler we are replacing a method of the parent class; a typo in the method name becomes a compile error.
`doGet(...) / doPost(...)`	Called by Tomcat (through service()) for GET and POST requests respectively.
`getServletInfo()`	Optional: returns a short description of the servlet.

3.5 Deployment descriptor: WEB-INF/web.xml

This is the complete web.xml for the project. It maps all three servlets used in Programs 1 and 2.

Web Pages/WEB-INF/web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="3.1" xmlns="http://xmlns.jcp.org/xml/ns/javaee"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
         http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd">
    <servlet>
        <servlet-name>RequestServlet</servlet-name>
        <servlet-class>RequestServlet</servlet-class>
    </servlet>
    <servlet>
        <servlet-name>PersonalInfoFrom</servlet-name>
        <servlet-class>PersonalInfoFrom</servlet-class>
    </servlet>
    <servlet>
        <servlet-name>PersonalInfo</servlet-name>
        <servlet-class>PersonalInfo</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>RequestServlet</servlet-name>
        <url-pattern>/RequestServlet</url-pattern>
    </servlet-mapping>
    <servlet-mapping>
        <servlet-name>PersonalInfoFrom</servlet-name>
        <url-pattern>/PersonalInfoFrom</url-pattern>
    </servlet-mapping>
    <servlet-mapping>
        <servlet-name>PersonalInfo</servlet-name>
        <url-pattern>/info</url-pattern>
    </servlet-mapping>
    <session-config>
        <session-timeout>30</session-timeout>
    </session-config>
</web-app>
```

Code / syntax	What it means
`<web-app version="3.1" ...>`	Root element; the namespace and schema tell Tomcat which servlet specification the file follows.
`<servlet>`	Declares a servlet: gives the Java class a logical name.
`<servlet-name>`	The logical name (any text). Must be identical in <servlet> and <servlet-mapping>.
`<servlet-class>`	Fully qualified class name. With a package it would be e.g. com.lab.RequestServlet.
`<servlet-mapping>`	Connects a servlet name to a URL pattern.
`<url-pattern>/info</url-pattern>`	The URL (after the context path) that triggers the servlet: http://localhost:8080/ServletLab/info.
`<session-timeout>30</session-timeout>`	An HttpSession expires after 30 minutes without requests.

TIP Instead of web.xml you can annotate the class with `@WebServlet("/RequestServlet")`. Use one approach per servlet, not both, or Tomcat reports a duplicate mapping.

3.6 Run the program

1. Save all files (**Ctrl+Shift+S**).
2. Right-click the project → **Clean and Build**. The Output window should end with **BUILD SUCCESSFUL**.
3. Right-click the project → **Run**. NetBeans starts Tomcat, deploys the app and opens `http://localhost:8080/ServletLab/`.
4. In the browser address bar open `http://localhost:8080/ServletLab/RequestServlet`.

3.7 Expected output

Browser (verified output; your date and IP will differ)

```
Current Date & Time: Mon Nov 09 16:07:34 IST 2026
Request Information Example
Method: GET
Request URI: /ServletLab/RequestServlet
Protocol: HTTP/1.1
PathInfo: null
Remote Address: 127.0.0.1
```

3.8 Try this

- Change the mapping to `/RequestServlet/*`, then open `/ServletLab/RequestServlet/abc`. PathInfo now shows `/abc`.
- Print `request.getHeader("User-Agent")` to see which browser made the request.
- Session 1 exercise 1: also print `System.currentTimeMillis()` as a timestamp.

4. Program 2: Capture a form with two Servlets

4.1 Goal

Servlet **PersonalInfoFrom** sends a Bootstrap-styled HTML form to the browser. When the user clicks **Submit**, the form is POSTed to `/info`, where servlet **PersonalInfo** reads every field with `request.getParameter()` and displays the values in a table. This is Example 2 of the manual (Session 1 exercise 2).

Request flow

```
Browser ──GET /PersonalInfoFrom──▶ PersonalInfoFrom.doGet()  ──▶ HTML form
Browser ──POST /info (form data)──▶ PersonalInfo.doPost()    ──▶ HTML table of values
```

4.2 Create the servlets

1. In the same ServletLab project: right-click **Source Packages** → **New** → **Servlet…**, name `PersonalInfoFrom`, URL pattern `/PersonalInfoFrom`, tick *Add information to web.xml* → Finish.
2. Repeat for `PersonalInfo` with URL pattern `/info`.
3. Replace the generated code with the listings below. The web.xml from 3.5 already contains both mappings.

NOTE The manual builds the HTML with one very long concatenated string. The listings below produce the same page but use one `out.println` per element so they are easier to read and debug. Single quotes inside the HTML avoid escaping double quotes (\").

4.3 Source code: PersonalInfoFrom.java (shows the form)

Source Packages/PersonalInfoFrom.java

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

// Servlet 1: sends the HTML form to the browser
public class PersonalInfoFrom extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        try (PrintWriter out = response.getWriter()) {
            out.println("<!DOCTYPE html><html lang='en'><head><meta charset='utf-8'>");
            out.println("<meta name='viewport' content='width=device-width, initial-scale=1'>");
            out.println("<link rel='stylesheet'
href='https://cdn.jsdelivr.net/npm/bootstrap@5.1.0/dist/css/bootstrap.min.css'>");
            out.println("<title>Servlet Lab</title></head><body><div class='container'>");
            out.println("<h1 class='text-center'>Personal Information Form</h1>");
            out.println("<form class='row g-3' action='info' method='post'>");
            out.println(" <div class='col-12'><label class='form-label'>Name</label>"
                    + "<input type='text' name='name' class='form-control'></div>");
            out.println(" <div class='col-md-6'><label class='form-label'>Email</label>"
                    + "<input type='email' name='emailId' class='form-control'></div>");
            out.println(" <div class='col-md-6'><label class='form-label'>Password</label>"
                    + "<input type='password' name='pwd' class='form-control'></div>");
            out.println(" <div class='col-md-6'><label class='form-label'>City</label>"
                    + "<input type='text' name='city' class='form-control'></div>");
            out.println(" <div class='col-md-4'><label class='form-label'>State</label>"
                    + "<select name='state' class='form-select'><option>Delhi</option>"
                    + "<option>Bihar</option><option>UP</option></select></div>");
            out.println(" <div class='col-md-2'><label class='form-label'>Zip</label>"
                    + "<input type='text' name='zip' class='form-control'></div>");
            out.println(" <div class='col-12 text-center'>"
                    + "<button type='submit' class='btn btn-primary'>Submit</button></div>");
            out.println("</form></div></body></html>");
        }
    }
}
```

4.4 Source code: PersonalInfo.java (handles the submission)

Source Packages/PersonalInfo.java

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

// Servlet 2: receives the submitted form (POST /info) and shows the values
public class PersonalInfo extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        String name  = request.getParameter("name");
        String email = request.getParameter("emailId");
        String city  = request.getParameter("city");
        String state = request.getParameter("state");
        String zip  = request.getParameter("zip");

        try (PrintWriter out = response.getWriter()) {
            out.println("<!DOCTYPE html><html lang='en'><head><meta charset='utf-8'>");
            out.println("<link rel='stylesheet'
href='https://cdn.jsdelivr.net/npm/bootstrap@5.1.0/dist/css/bootstrap.min.css'>");
            out.println("<title>Servlet Lab</title></head><body><div class='container'>");
            out.println("<h1 class='text-center'>Personal Information</h1>");
            out.println("<table class='table'>");
            out.println("<tr><th>Name</th><td>"  + name  + "</td></tr>");
            out.println("<tr><th>Email</th><td>" + email + "</td></tr>");
            out.println("<tr><th>City</th><td>"  + city  + "</td></tr>");
            out.println("<tr><th>State</th><td>" + state + "</td></tr>");
            out.println("<tr><th>Zip</th><td>"  + zip  + "</td></tr>");
            out.println("</table></div></body></html>");
        }
    }
}
```

4.5 Syntax explained

Code / syntax	What it means
`<link rel='stylesheet' href='...bootstrap.min.css'>`	Loads Bootstrap's CSS from a CDN (needs internet). It only changes the look of the page.
`<form action='info' method='post'>`	On submit, the browser sends the fields to the relative URL info (= /ServletLab/info) using POST.
`<input type='text' name='name'>`	A text box. The name attribute becomes the parameter name on the server.
`<input type='email'> / type='password'`	Email boxes get browser validation; password boxes hide what is typed.
`<select name='state'><option>…`	Drop-down list. The selected option's text (or value attribute) is sent.
`class='row g-3', 'col-md-6'`	Bootstrap grid: a row of 12 columns; col-md-6 takes half the width on medium and larger screens; g-3 adds gaps.
`class='form-control' / 'form-select'`	Bootstrap styling for inputs and drop-downs (full width, rounded borders).
`class='btn btn-primary'`	Bootstrap blue button.
`request.getParameter("emailId")`	Returns the value typed into the field whose name is emailId, as a String; null if no such field was sent.
`doPost(...) only`	PersonalInfo overrides only doPost, so opening /info directly in the address bar (a GET) gives HTTP 405 Method Not Allowed. That is expected.

IMPORTANT The string in `getParameter("...")` must match the input's `name` attribute exactly, including upper/lower case. A mismatch is the number-one cause of *null* values.

4.6 Run the program

1. Run the project (it is redeployed automatically).
2. Open `http://localhost:8080/ServletLab/PersonalInfoFrom`.
3. Fill in the form and click **Submit**. The URL changes to `/ServletLab/info` and the table appears.

4.7 Expected output

Browser after submitting (verified)

```
Personal Information
Name  Asha
Email  a@x.in
City  Pune
State  UP
Zip    411001
```

4.8 Try this: GET vs POST

Change `method='post'` to `method='get'` in PersonalInfoFrom and rename `doPost` to `doGet` in PersonalInfo. Submit again and look at the address bar:

Address bar with GET

```
http://localhost:8080/ServletLab/info?name=Asha&emailId=a%40x.in&pwd=secret&city=Pune&state=UP&zip=411001
```

With GET every field, including the password, is visible in the URL, browser history and server logs. That is why login and registration forms always use POST.

	GET	POST
Data location	URL query string	Request body
Visible / bookmarkable	Yes	No
Length limit	Limited by URL length	No practical limit
Typical use	Search, filters, links	Login, registration, saving data
Servlet method	doGet()	doPost()

5. Program 3: The same form with JSP

5.1 Goal

Rebuild Program 2 with two JSP pages. `stdForm.jsp` is plain HTML; `studentView.jsp` uses JSP expressions to print the submitted values, the entry date and the client IP. Notice how much less Java is needed.

5.2 How a JSP runs

1. The first time a JSP is requested, Tomcat **translates** it into a Java servlet source file (e.g. `studentView_jsp.java`).
2. Tomcat **compiles** that file into a .class file.
3. Tomcat **runs** it like any servlet: `jspInit()` once, `_jspService()` for every request, `jspDestroy()` at shutdown.
4. Later requests reuse the compiled class until the .jsp file changes.

5.3 Create the pages

1. Use the ServletLab project or create a new Web Application (File → New Project → Java with Ant → Java Web → Web Application → name → server → Finish).
2. Right-click **Web Pages** → **New** → **JSP…**, File Name `stdForm` → Finish. Replace the content with the listing below.
3. Repeat for `studentView`.
4. No web.xml entries are needed: a JSP is reached directly by its file name.

5.4 Source code: stdForm.jsp

Web Pages/stdForm.jsp

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css">
  <title>JSP Project</title>
</head>
<body>
<div class="container">
  <h1 class="text-center">Personal Information Form</h1>
  <form class="row g-3" action="studentView.jsp" method="post">
    <div class="col-12">
      <label class="form-label">Name</label>
      <input type="text" name="name" class="form-control">
    </div>
    <div class="col-md-6">
      <label class="form-label">Email</label>
      <input type="email" name="emailId" class="form-control">
    </div>
    <div class="col-md-6">
      <label class="form-label">City</label>
      <input type="text" name="city" class="form-control">
    </div>
    <div class="col-md-4">
      <label class="form-label">State</label>
      <select name="state" class="form-select">
        <option value="Delhi" selected>Delhi</option>
        <option value="Bihar">Bihar</option>
        <option value="UP">UP</option>
      </select>
    </div>
    <div class="col-md-2">
      <label class="form-label">Zip</label>
      <input type="text" name="zipCode" class="form-control">
    </div>
    <div class="col-12 text-center">
      <button type="submit" class="btn btn-primary">Submit</button>
    </div>
  </form>
</div>
</body>
</html>
```

5.5 Source code: studentView.jsp

Web Pages/studentView.jsp

```
<%@page import="java.util.Date"%>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css">
  <title>JSP Project: View Personal Info</title>
</head>
<body>
<div class="container">
  <h1 class="text-center">Personal Information</h1>
  <table class="table">
    <tr><th>Name:</th>      <td><%= request.getParameter("name") %></td></tr>
    <tr><th>Email Id:</th>  <td><%= request.getParameter("emailId") %></td></tr>
    <tr><th>City:</th>      <td><%= request.getParameter("city") %></td></tr>
    <tr><th>State:</th>      <td><%= request.getParameter("state") %></td></tr>
    <tr><th>Zip Code:</th>  <td><%= request.getParameter("zipCode") %></td></tr>
    <tr><th>Entry Date:</th> <td><%= new Date() %></td></tr>
    <tr><th>IP Address:</th> <td><%= request.getRemoteAddr() %></td></tr>
  </table>
</div>
</body>
</html>
```

5.6 Syntax explained

Code / syntax	What it means
`<%@page contentType="text/html" pageEncoding="UTF-8"%>`	Page directive: the response is HTML and the .jsp file itself is saved as UTF-8.
`<%@page import="java.util.Date"%>`	Page directive that imports a Java class so we can write new Date() instead of new java.util.Date().
`action="studentView.jsp"`	The form posts to the other JSP page in the same folder.
`<%= request.getParameter("name") %>`	Expression tag: evaluates the Java expression and writes the result into the page. No semicolon inside.
`request`	Implicit object of type HttpServletRequest: available in every JSP without declaring it.
`<%= new Date() %>`	Prints the current date/time (the Entry Date).
`<%= request.getRemoteAddr() %>`	Prints the client's IP address.
`<%-- comment --%>`	JSP comment: removed on the server, never sent to the browser (unlike <!-- -->).

Other JSP elements you will need in Session 2

Element	Syntax	Example
Scriptlet	<% java statements; %>	<% for (int i = 1; i <= 3; i++) { %> Row <%= i %> <% } %>
Declaration	<%! declaration %>	<%! int hits = 0; %> then <%= ++hits %>
Include directive	<%@ include file="x.jsp" %>	Copies the file in at translation time
Include action	<jsp:include page="x.jsp"/>	Includes the output at request time
Forward action	<jsp:forward page="y.jsp"/>	Hands the request to another page
useBean	<jsp:useBean id="s" class="lab.Student"/>	With <jsp:setProperty name="s" property="*"/>
JSTL	<%@ taglib prefix="c" uri="…/jsp/jstl/core" %>	<c:if>, <c:forEach>, <c:choose>, <c:out>, <c:url>, <c:redirect>

Implicit object	Type	Typical use
request	HttpServletRequest	Form parameters, headers, client IP
response	HttpServletResponse	sendRedirect(), headers, cookies
out	JspWriter	out.println() inside scriptlets
session	HttpSession	Store the logged-in user: session.setAttribute("user", n)
application	ServletContext	Data shared by all users (e.g. visitor counter)
pageContext	PageContext	Access attributes in any scope
config	ServletConfig	Init parameters from web.xml
page	Object	The current JSP instance (like this)
exception	Throwable	Only in pages with <%@ page isErrorPage="true" %>

5.7 Run the program

1. Right-click `stdForm.jsp` → **Run File**, or open `http://localhost:8080/ServletLab/stdForm.jsp`.
2. Fill the form and click **Submit**.

Expected output (verified)

```
Personal Information
Name:        Ravi
Email Id:    r@x.in
City:        Patna
State:      Bihar
Zip Code:    800001
Entry Date:  Mon Nov 09 16:07:34 IST 2026
IP Address:  127.0.0.1
```

5.8 Try this

- Change the form to `method="get"`: the page still works (JSP handles both) and the data appears in the URL (manual Figure 2.36).
- Add `<meta http-equiv="refresh" content="5">` to a JSP that prints `<%= new Date() %>`: an auto-refreshing clock (Session 2 exercise 1).

6. Program 4: Spring Boot MVC form with Thymeleaf and Bootstrap

6.1 Goal

Build the **StudentAdmission** application: a dashboard page, a student form bound to a Java object, and a controller that receives the form, adds the entry date and client IP, and shows the result on the dashboard. This is section 2.6 / 2.6.2 of the manual and the base project for Sessions 3–10.

6.2 Generate the project with Spring Initializr

1. Open **https://start.spring.io**.
2. Fill in the fields as in the table below.
3. Click **ADD DEPENDENCIES…** and add **Spring Web**, **Thymeleaf** and **Spring Boot DevTools**.
4. Click **GENERATE**. A file `StudentAdmission.zip` downloads.
5. Extract it into your Eclipse workspace folder.

Field	Value	Meaning
Project	Maven	Build tool that downloads libraries listed in pom.xml
Language	Java	
Spring Boot	Latest stable (default)	Tested with 4.1.2
Group	com.ignou.lab.webtechnologies	Reverse domain name of the organisation
Artifact / Name	StudentAdmission	Project and jar/war name
Description	Lab Project for Student Admission using Spring Boot	
Package name	com.ignou.lab.webtechnologies.studentadmission	Base Java package
Packaging	War (Jar also works)	War can also be deployed to an external Tomcat
Java	17	Must not be higher than your installed JDK

IMPORTANT Add **Spring Data JPA** and **MySQL Driver** only when you need a database (Session 6). With JPA on the classpath and no `spring.datasource.*` settings, the app stops with *Failed to configure a DataSource* (manual Figure 2.45).

6.3 Open the project in Eclipse

1. **File → Open Projects from File System… → Directory… →** select the extracted `StudentAdmission` folder **→ Finish**.
2. Wait while Maven downloads all dependencies (progress bar at the bottom right). Internet is required the first time.
3. If you later edit pom.xml and Eclipse shows errors: right-click the project **→ Maven → Update Project… →** tick *Force Update of Snapshots/Releases* → OK.

6.4 Final project structure

Directory layout

```
StudentAdmission
├── pom.xml
└── src/main
    ├── java/com/ignou/lab/webtechnologies/studentadmission
    │   ├── StudentAdmissionApplication.java  (generated: main class)
    │   ├── ServletInitializer.java            (generated: for war packaging)
    │   ├── controller/StudentController.java  (you create)
    │   └── entity/Student.java                (you create)
    └── resources
        ├── application.properties
        ├── static/css/master.css              (you create)
        └── templates
            ├── dashboard.html                 (you create)
            └── stdForm.html                   (you create)
```

Create packages with right-click on the base package → **New** → **Package** (`…studentadmission.controller` and `…studentadmission.entity`), classes with **New** → **Class**, and HTML files with **New** → **File** inside `src/main/resources/templates`.

6.5 pom.xml

Generated by Initializr; the four WebJars dependencies (Bootstrap, jQuery, Font Awesome and the locator) were added by hand.

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.1.2</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.ignou.lab.webtechnologies</groupId>
    <artifactId>StudentAdmission</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <packaging>war</packaging>
    <name>StudentAdmission</name>
    <description>Lab Project for Student Admission using Spring Boot</description>
    <properties>
        <java.version>17</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-thymeleaf</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc</artifactId>
        </dependency>

        <!-- Bootstrap, jQuery and Font Awesome packaged as WebJars -->
        <dependency>
            <groupId>org.webjars</groupId>
            <artifactId>webjars-locator-lite</artifactId>
        </dependency>
        <dependency>
            <groupId>org.webjars</groupId>
            <artifactId>jquery</artifactId>
            <version>3.7.1</version>
        </dependency>
        <dependency>
            <groupId>org.webjars</groupId>
            <artifactId>bootstrap</artifactId>
            <version>5.3.3</version>
        </dependency>
```

pom.xml (continued)

```
        <dependency>
            <groupId>org.webjars</groupId>
            <artifactId>font-awesome</artifactId>
            <version>6.5.2</version>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-tomcat</artifactId>
            <scope>provided</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-thymeleaf-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>

</project>
```

Code / syntax	What it means
`<parent> spring-boot-starter-parent`	Inherits sensible defaults and the tested version of every Spring library, so most dependencies need no <version>.
`<groupId>, <artifactId>, <version>`	The project's coordinates (identity) in the Maven world.
`<packaging>war</packaging>`	Build a .war file. It still runs stand-alone with java -jar because Tomcat is embedded.
`<java.version>17</java.version>`	Compile for Java 17.
`spring-boot-starter-webmvc`	Spring MVC + embedded Tomcat + JSON support. (Spring Boot 2.x/3.x call it spring-boot-starter-web.)
`spring-boot-starter-thymeleaf`	The Thymeleaf template engine for the HTML views.
`webjars-locator-lite + org.webjars:*`	Bootstrap, jQuery and Font Awesome packaged as jars and served at /webjars/... The locator lets the URLs omit version numbers. (Boot 2.x: artifact webjars-locator, as in the manual.)
`spring-boot-devtools (runtime, optional)`	Automatic restart when you save a file; never included in production builds.
`spring-boot-starter-tomcat (provided)`	Tomcat is available when running, but not bundled when deploying the war to an external Tomcat.
`<scope>test</scope>`	Only used when running unit tests.

Code / syntax	What it means
`spring-boot-maven-plugin`	Builds an executable jar/war and provides mvn spring-boot:run.

6.6 application.properties

src/main/resources/application.properties

```
spring.application.name=StudentAdmission
# ================================
# Server properties
# ================================
server.port=8080
server.servlet.context-path=/
```

Every line is `key=value`; lines starting with `#` are comments. `server.port` changes the port (e.g. 8082 if 8080 is busy). `server.servlet.context-path=/` means the app is at the root: http://localhost:8080/.

When you add a database in Session 6, append the settings from the manual:

Database settings (Session 6 onwards)

```
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/Student_Admission
spring.datasource.username=root
spring.datasource.password=your_password
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```

6.7 The Model: entity/Student.java

entity/Student.java

```
package com.ignou.lab.webtechnologies.studentadmission.entity;

import java.util.Date;

public class Student {

    private int id;
    private String name;
    private String emailId;
    private String password;
    private String address1;
    private String address2;
    private String city;
    private String state;
    private String zipCode;
    private Date entryDate;
    private String ip;

    public Student() {
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmailId() { return emailId; }
    public void setEmailId(String emailId) { this.emailId = emailId; }

    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }

    public String getAddress1() { return address1; }
    public void setAddress1(String address1) { this.address1 = address1; }

    public String getAddress2() { return address2; }
    public void setAddress2(String address2) { this.address2 = address2; }

    public String getCity() { return city; }
    public void setCity(String city) { this.city = city; }

    public String getState() { return state; }
    public void setState(String state) { this.state = state; }
```

entity/Student.java (continued)

```
    public String getZipCode() { return zipCode; }
    public void setZipCode(String zipCode) { this.zipCode = zipCode; }

    public Date getEntryDate() { return entryDate; }
    public void setEntryDate(Date entryDate) { this.entryDate = entryDate; }

    public String getIp() { return ip; }
    public void setIp(String ip) { this.ip = ip; }
}
```

Code / syntax	What it means
`package ...entity;`	Must match the folder the file is in.
`private String name;`	Fields are private (encapsulation); other classes use getters and setters.
`public Student() { }`	No-argument constructor. Spring needs it to create an empty Student before filling it from the form.

Code / syntax	What it means
`getName() / setName(...)`	JavaBean accessors. Thymeleaf reads values through getters; Spring's data binding writes form values through setters. Names must follow the getXxx/setXxx pattern exactly.
`Date entryDate, String ip`	Not on the form; the controller fills them in.

TIP In Eclipse you don't have to type getters and setters: right-click inside the class → **Source** → **Generate Getters and Setters…** → Select All → Generate.

6.8 The Controller: controller/StudentController.java

controller/StudentController.java

```
package com.ignou.lab.webtechnologies.studentadmission.controller;

import com.ignou.lab.webtechnologies.studentadmission.entity.Student;
import jakarta.servlet.http.HttpServletRequest;
import java.util.Date;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class StudentController {

    @GetMapping("/")
    public String welcome() {
        return "redirect:/dashboard";
    }

    @GetMapping("/dashboard")
    public String mainPage() {
        return "dashboard";
    }

    @GetMapping("/stdForm")
    public String studentInputForm(Model theModel) {
        Student student = new Student();
        theModel.addAttribute("student", student);
        return "stdForm";
    }

    @PostMapping("/student/view")
    public String viewStdInfo(@ModelAttribute("student") Student theStudent,
                              Model theModel, HttpServletRequest request) {
        theStudent.setEntryDate(new Date());
        theStudent.setIp(request.getRemoteAddr());
        theModel.addAttribute("student", theStudent);
        return "dashboard";
    }
}
```

IMPORTANT The manual labels this listing *Student.java (Controller Class)*, but the class is named StudentController, so the file must be **StudentController.java**. Also, with Spring Boot 3 or later the import is `jakarta.servlet.http.HttpServletRequest` (Boot 2.x: javax).

Code / syntax	What it means
`@Controller`	Marks the class as a Spring MVC controller. Spring finds it automatically because it is inside the main application's package.

Code / syntax	What it means
`@GetMapping("/")`	This method handles HTTP GET requests for the URL /.
`return "redirect:/dashboard"`	Sends HTTP 302 to the browser, which then requests /dashboard. URL in the address bar changes.
`return "dashboard"`	A view name. Thymeleaf renders src/main/resources/templates/dashboard.html.
`Model theModel`	A map of data handed to the view. Spring creates it and passes it in.
`theModel.addAttribute("student", new Student())`	Puts an empty Student into the model under the name student; the form binds to it with th:object="${student}".
`@PostMapping("/student/view")`	Handles the POST sent by the form's th:action="@{/student/view}".
`@ModelAttribute("student") Student theStudent`	Data binding: Spring creates a Student and calls setName(), setEmailId(), … for each form field whose name matches a property.
`HttpServletRequest request`	The same request object as in servlets; any controller method can ask for it.
`setEntryDate(new Date()), setIp(...)`	Add server-side information that the user did not type.

6.9 The Views: templates/stdForm.html and templates/dashboard.html

src/main/resources/templates/stdForm.html

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <title>Student Information Form</title>
  <link th:href="@{/webjars/bootstrap/css/bootstrap.min.css}" rel="stylesheet">
  <link th:href="@{/webjars/font-awesome/css/all.css}" rel="stylesheet">
  <link th:href="@{/css/master.css}" rel="stylesheet">
</head>
<body>
<div class="container">
  <h1 class="text-center">Personal Information Form</h1>
  <form class="row g-3" th:action="@{/student/view}" th:object="${student}" method="post">
    <div class="col-12">
      <label class="form-label">Name</label>
      <input type="text" th:field="*{name}" class="form-control">
    </div>
    <div class="col-md-6">
      <label class="form-label">Email</label>
      <input type="email" th:field="*{emailId}" class="form-control">
    </div>
    <div class="col-md-6">
      <label class="form-label">Password</label>
      <input type="password" th:field="*{password}" class="form-control">
    </div>
    <div class="col-12">
      <label class="form-label">Address</label>
      <input type="text" th:field="*{address1}" class="form-control">
    </div>
    <div class="col-12">
      <label class="form-label">Address 2</label>
      <input type="text" th:field="*{address2}" class="form-control">
    </div>
    <div class="col-md-6">
      <label class="form-label">City</label>
      <input type="text" th:field="*{city}" class="form-control">
    </div>
    <div class="col-md-4">
      <label class="form-label">State</label>
      <select th:field="*{state}" class="form-select">
        <option value="Delhi">Delhi</option>
        <option value="Bihar">Bihar</option>
        <option value="UP">UP</option>
      </select>
```

src/main/resources/templates/stdForm.html (continued)

```
    </div>
    <div class="col-md-2">
      <label class="form-label">Zip</label>
      <input type="text" th:field="*{zipCode}" class="form-control">
    </div>
    <div class="col-12 text-center">
      <button type="submit" class="btn btn-primary"><i class="fas fa-save"></i> Submit</button>
    </div>
  </form>
</div>
<script th:src="@{/webjars/jquery/jquery.min.js}"></script>
<script th:src="@{/webjars/bootstrap/js/bootstrap.bundle.min.js}"></script>
</body>
</html>
```

src/main/resources/templates/dashboard.html

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <title>Student Admission Dashboard</title>
  <link th:href="@{/webjars/bootstrap/css/bootstrap.min.css}" rel="stylesheet">
  <link th:href="@{/webjars/font-awesome/css/all.css}" rel="stylesheet">
  <link th:href="@{/css/master.css}" rel="stylesheet">
</head>
<body>
<div class="container">
  <h1 class="text-center">Lab Project for Student Admission using Spring Boot</h1>
  <div class="text-center mt-4">
    <a class="btn btn-primary" th:href="@{/stdForm}">
      Student Form <i class="fas fa-arrow-right"></i></a>
  </div>
  <table class="table mt-4">
    <thead><tr><th colspan="2">Student Information Via Controller</th></tr></thead>
    <tbody>
      <tr><th>Name:</th>      <td th:text="${student?.name}"></td></tr>
      <tr><th>Email Id:</th>  <td th:text="${student?.emailId}"></td></tr>
      <tr><th>Address:</th>    <td th:text="${student?.address1}"></td></tr>
      <tr><th>Address 2:</th>  <td th:text="${student?.address2}"></td></tr>
      <tr><th>City:</th>      <td th:text="${student?.city}"></td></tr>
      <tr><th>State:</th>      <td th:text="${student?.state}"></td></tr>
      <tr><th>Zip Code:</th>  <td th:text="${student?.zipCode}"></td></tr>
      <tr><th>Entry Date:</th> <td th:text="${student?.entryDate}"></td></tr>
      <tr><th>IP Address:</th> <td th:text="${student?.ip}"></td></tr>
    </tbody>
  </table>
</div>
<script th:src="@{/webjars/jquery/jquery.min.js}"></script>
<script th:src="@{/webjars/bootstrap/js/bootstrap.bundle.min.js}"></script>
</body>
</html>
```

src/main/resources/static/css/master.css

```
body { background: #f7f9fc; padding-top: 24px; }
h1  { color: #1b2a41; margin-bottom: 24px; }
```

Code / syntax	What it means
`xmlns:th="http://www.thymeleaf.org"`	Declares the th: prefix so the IDE recognises Thymeleaf attributes.
`th:href="@{/webjars/bootstrap/css/bootstrap.min.css}"`	@{...} is a link expression: Thymeleaf builds the URL and adds the context path if there is one.
`th:action="@{/student/view}"`	Form target URL, matched by @PostMapping("/student/view").
`th:object="${student}"`	Selects the model attribute student as the form-backing object.
`th:field="*{name}"`	*{...} is a selection expression on th:object. Generates id="name" name="name" value="(current value)".
`<select th:field="*{state}">`	Also marks the option matching the current value as selected.
`th:text="${student?.name}"`	Replaces the cell's content with the value. ?. is safe navigation: if student is null (first visit to /dashboard) nothing is printed instead of throwing SpelEvaluationException.
`th:src="@{/webjars/jquery/jquery.min.js}"`	Loads jQuery from the WebJar. Scripts go just before </body>.
`<i class="fas fa-save"></i>`	A Font Awesome icon (solid style, floppy-disk).
`static/css/master.css`	Files in src/main/resources/static are served as-is, at /css/master.css.

6.10 Run the application

From Eclipse

1. Right-click the project → **Run As** → **Spring Boot App** (or open StudentAdmissionApplication.java → Run As → Java Application).
2. Watch the Console. Success looks like the lines below.
3. Open `http://localhost:8080/` in the browser.

Console (verified)

```
Tomcat started on port 8080 (http) with context path '/'
Started StudentAdmissionApplication in 1.377 seconds
```

From the command line (any OS)

Terminal

```
cd StudentAdmission
./mvnw spring-boot:run          # Linux / macOS
mvnw.cmd spring-boot:run        # Windows

# or build and run the war
./mvnw package
java -jar target/StudentAdmission-0.0.1-SNAPSHOT.war
```

URLs to test

URL	Handled by	Result
GET /	welcome()	Redirects (HTTP 302) to /dashboard
GET /dashboard	mainPage()	Dashboard with an empty table and a Student Form button
GET /stdForm	studentInputForm()	The Personal Information Form
POST /student/view	viewStdInfo()	Dashboard filled with the submitted values, entry date and IP

6.11 Expected output

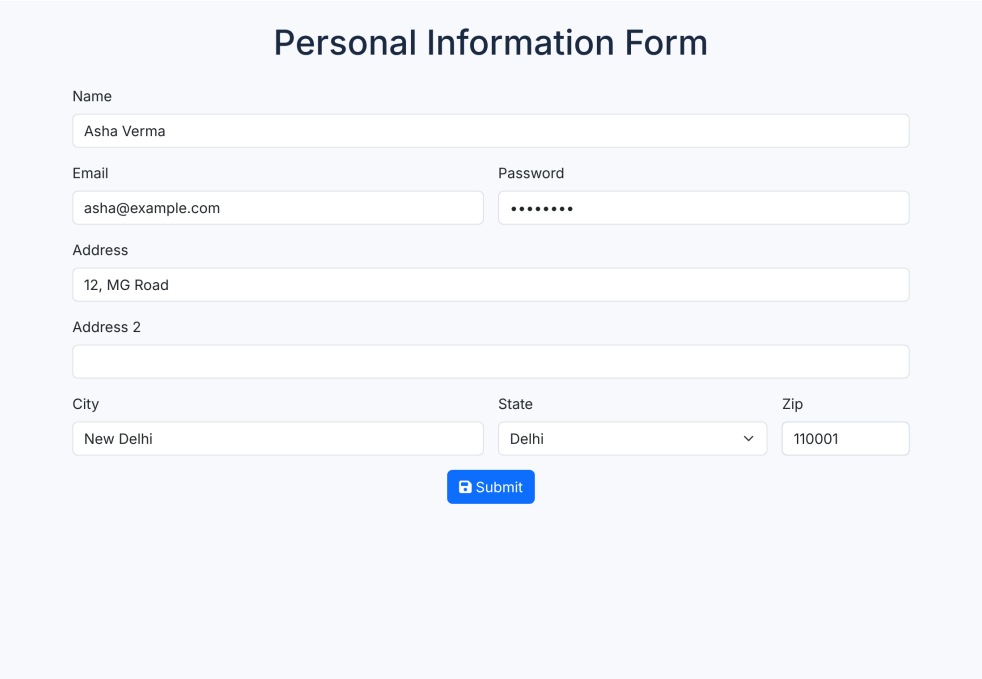


Figure 1: http://localhost:8080/stdForm, filled in (screenshot from the tested project)

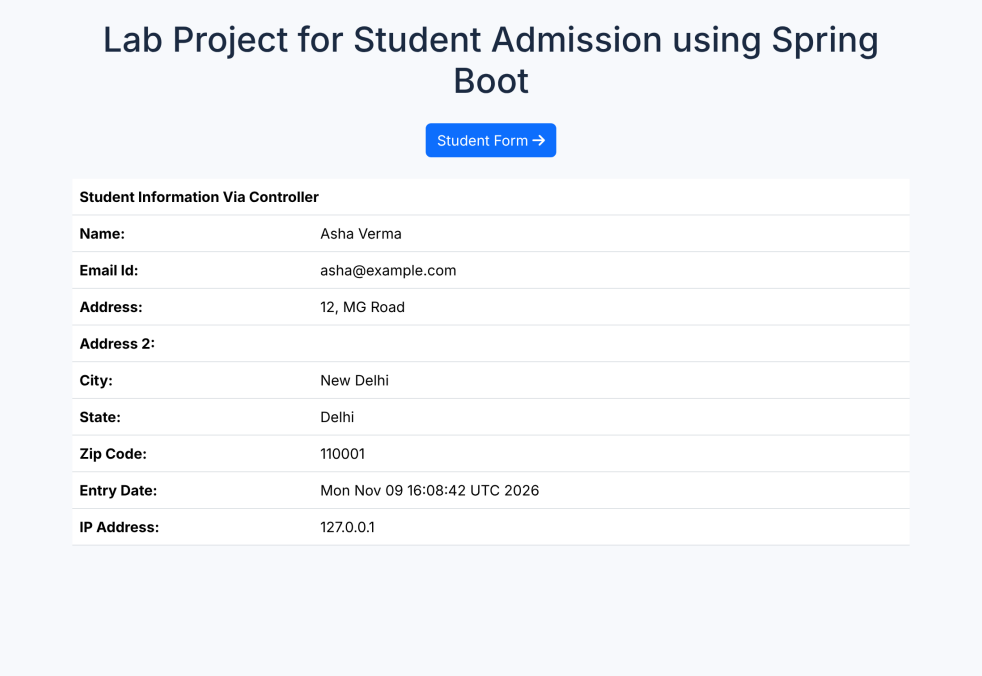


Figure 2: after Submit, POST /student/view returns the dashboard with the bound values

6.12 Try this

- Change a template and save: DevTools restarts the app; refresh the browser to see the change.
- Add a `<textarea th:field="*{address2}">` and a checkbox bound to a new `boolean hostel` field (Session 4 exercise 4).
- Add `required` and `pattern="[0-9]{6}"` to the Zip input for client-side validation (Session 5 exercise 1).

7. Adding Bootstrap, jQuery and Font Awesome

7.1 Method 1: WebJars through Maven (works offline)

1. Add the dependencies to pom.xml (exactly as in 6.5).
2. Right-click project → **Maven** → **Update Project…** → OK.
3. Put the CSS links between `<head>` and `</head>`, and the scripts just before `</body>`.

HTML (Thymeleaf)

```
<!-- in <head> -->
<link th:href="@{/webjars/bootstrap/css/bootstrap.min.css}" rel="stylesheet">
<link th:href="@{/webjars/font-awesome/css/all.css}" rel="stylesheet">

<!-- just before </body> -->
<script th:src="@{/webjars/jquery/jquery.min.js}"></script>
<script th:src="@{/webjars/bootstrap/js/bootstrap.bundle.min.js}"></script>
```

In a plain (non-Thymeleaf) HTML file use `href="/webjars/..."` and `src="/webjars/..."` instead of th:href / th:src.

7.2 Method 2: CDN links (needs internet when the page loads)

Copy the **Starter template** from **getbootstrap.com** → **Docs** → **Getting started**. Its essential parts:

index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
  <title>Hello, world!</title>
</head>
<body>
  <h1>Hello, world!</h1>
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

For Font Awesome via CDN, register at **fontawesome.com** to get a personal *kit* script (`<script src="https://kit.fontawesome.com/xxxx.js" crossorigin="anonymous"></script>`) and place it in `<head>`. Then use icons such as `<i class="fas fa-thumbs-up fa-5x"></i>`.

7.3 Bootstrap classes used in the lab

Class	Effect
`container`	Centred, responsive page width
`row, col-12, col-md-6, col-md-4, col-md-2`	12-column grid; md = applies from medium screens up
`g-3, mt-4`	Gutter (gap) between columns; margin-top
`form-label, form-control, form-select`	Styled labels, inputs and drop-downs
`btn btn-primary`	Primary (blue) button
`table, table-responsive`	Styled table; horizontal scroll on small screens
`text-center, text-primary`	Centre text; blue text

8. Troubleshooting

Symptom	Likely cause	Fix
'java' is not recognized as an internal or external command	PATH does not contain the JDK bin folder, or the Command Prompt was opened before the change	Add %JAVA_HOME%\bin to Path (2.3) and open a new Command Prompt
NetBeans: No servers are registered	Tomcat not added	Tools → Servers → Add Server (2.5)
HTTP 404 – Not Found	URL does not match url-pattern / @GetMapping, wrong context path, or wrong case	Compare the URL with web.xml or the annotation character by character
HTTP 405 – Method Not Allowed	Servlet only overrides doPost but the browser sent GET (or vice versa)	Use the matching form method, or implement both doGet and doPost
HTTP 500 / ClassNotFoundException: javax.servlet...	javax code deployed to Tomcat 10+	Use Tomcat 9, or change imports to jakarta.servlet
Form values print as null	name attribute ≠ getParameter() string	Make them identical (case-sensitive)
Port 8080 was already in use	Tomcat from NetBeans or another app is running	Stop it, or set server.port=8082 in application.properties
Failed to configure a DataSource	Spring Data JPA dependency without database settings	Add spring.datasource.* properties or remove the JPA dependency
Communications link failure (MySQL)	MySQL not running, wrong port/database name	Start MySQL; check the URL jdbc:mysql://localhost:3306/Student_Admission
TemplateInputException: template might not exist	View name returned by the controller has no matching file	Place dashboard.html / stdForm.html in src/main/resources/templates
SpelEvaluationException: property 'name' cannot be found on null	Model attribute missing on first page load	Use the safe navigation operator: ${student?.name}
Bootstrap styles not applied	WebJar missing / Maven not updated / no internet for CDN	Maven → Update Project; open the CSS URL directly to check for 404
Eclipse: red errors right after import	Dependencies still downloading or outdated	Wait for the build; then Maven → Update Project (Force Update)
Changes not visible	Old deployment or browser cache	Clean and Build, redeploy; hard refresh with Ctrl+F5

9. Lab assignments (session-wise)

The exercises below are from section 2.7 of the lab manual. Use NetBeans or Eclipse. The programs in this handbook give you a working starting point for the first exercises of Sessions 1–5.

Session 1: Basics of Servlet

1. Write a Servlet program to print the current date and time along with the timestamp.
2. Create an HTML form with the input of student information using HTTP protocol and method, then display the input information using a Servlet.
3. Write a servlet program to capture client IPs and display them.
4. Write a servlet program for session management using HttpSession along with tracking, and also use a cookie for session tracking.
5. Write a CRUD (Create/Save, Read, Edit/Update, Delete) application using servlets. Create a database named IGNOU and a table named Student which captures student information (basics, contact, enrolment details along with courses). Make the necessary assumptions.

Session 2: JSP

1. Write a JSP program to print the current date and time along with a timestamp; implement auto-refresh of the page.
2. Create a JSP page and implement a scripting tag, expression tag and declaration tag.
3. Import the JSTL library in a JSP page and use its tags: (a) out (b) if (c) forEach (d) choose, when and otherwise (e) url and redirect.
4. Create a JSP page for database connectivity using JDBC and show the student details from the database created in Session 1 exercise 5.
5. Write a JSP application using the action elements: (a) jsp:forward (b) jsp:include (c) setProperty and getProperty (d) jsp:useBean.
6. Write a JSP program using the implicit objects out, request, response, session, pageContext and exception with an example.
7. Create a JSP project implementing all the Session 1 and Session 2 concepts: login form, CRUD of student details, session management with exception handling using Servlet and JSP.

Session 3: Maven, framework for J2EE

1. Create a Maven-based project using Spring Initializr.
2. Create a Maven-based J2EE application for Spring MVC.
3. Create a Maven-based J2EE application for Hibernate and JPA, using the database from Session 1 exercise 5.
4. Define a new implementation of a Teacher interface for his/her favourite course in Spring using Inversion of Control, and retrieve the information from the new teacher implementation.

Session 4: Dependency injection, controller

1. Write a class and implement it using dependency injection.
2. Write a service interface with a getRandom() method, define 3 courses in an array, and inject it using dependency injection into the Teacher interface (Session 3 exercise 4). Test and verify retrieval of random courses.
3. Write a program using Spring Framework to create a controller and display the response in a view using @GetMapping().

4. Create a form to capture student admission information using Spring form tags (text, textarea, dropdown, date picker, true/false and checkbox) and write a controller using @PostMapping() to display the form information.

Session 5: Form validation, Bootstrap and CSS

1. Apply client-side validation to the form from Session 4 exercise 4, along with server-side validation.
2. Write a program to bind form objects with an entity bean in Spring MVC.
3. Configure Bootstrap in Spring MVC and use default styling classes in the forms and views created above.
4. Apply custom styling to your pages in Spring MVC.

Session 6: Hibernate, JPA

1. Create a database for the student admission lifecycle and configure Hibernate and JPA with the Spring MVC project along with all table entities.
2. Retrieve student information using Hibernate and print the values in the console.
3. Create CRUD (Create/Save, Read/Fetch, Edit/Update, Delete) using Spring MVC and Hibernate.
4. Apply a batch update for students' admission approval using Spring MVC and Hibernate.

Session 7: Spring Boot and REST controller

1. Create a Spring Boot application using Spring Initializr. Add these dependencies manually: Spring MVC, Hibernate, JPA, Thymeleaf, DevTools, Actuator, and a MySQL/MSSQL/Oracle/MongoDB driver (your choice).
2. Configure database settings through the property file in Spring Boot.
3. Create JPA repositories for all entities used in the student admission lifecycle.
4. Create a REST controller to fetch student information using a JPA repository; the response should be displayed in JSON format.

Session 8: REST API, Spring Security, Actuator

1. Add the Actuator dependency in Spring Boot and test all available Actuator endpoints.
2. Create a REST API for CRUD operations using Spring Boot and Hibernate/JPA using annotations.
3. Create a REST API for CRUD operations using Spring Boot and Hibernate/JPA using XML configuration.
4. Add the Spring Security dependency to the application from Session 8 exercise 2 and configure the default login.

Session 9: Spring Security

1. Create a User table in the database and bind the user entity with Spring Security for login.
2. Create a custom login page in HTML and authenticate using Spring Security in Spring Boot.
3. Implement logout functionality in Spring Security.
4. Create a user registration form and validate it. Once the information is validated and saved, auto-login using Spring Security.

Session 10: Bootstrap, role-based authentication and CSRF

1. Add Bootstrap styling to the login and registration pages from Session 9.
2. Create a Role table in the database and configure it with Spring Boot, Security and Hibernate.
3. Write code for role-based authentication using Spring Security.
4. Display the current logged-in user's details on the dashboard along with client IP, date-time and the user's current role.
5. Add CSRF functionality to the authentication.

6. Restrict role-based access to views in Spring Boot Security.

10. Quick reference

10.1 Servlet API

Call	Returns / does
`request.getParameter("n")`	Value of one form field (String or null)
`request.getParameterValues("n")`	All values of a multi-select / checkbox group (String[])
`request.getSession()`	Current HttpSession (creates one if needed)
`session.setAttribute("k", v) / getAttribute("k")`	Store / read per-user data
`response.addCookie(new Cookie("k", "v"))`	Send a cookie to the browser
`request.getCookies()`	Cookies sent by the browser (Cookie[])
`response.sendRedirect("url")`	Tell the browser to load another URL
`request.getRequestDispatcher("x.jsp").forward(req, res)`	Hand the request to a JSP/servlet on the server

10.2 Spring MVC annotations

Annotation	Purpose
`@SpringBootApplication`	Main class: enables auto-configuration and component scanning
`@Controller`	Web controller that returns view names
`@RestController`	Controller whose return values are written as JSON (Session 7)
`@GetMapping / @PostMapping`	Map HTTP GET / POST on a URL to a method
`@RequestParam("n")`	Bind one request parameter to a method argument
`@PathVariable`	Bind part of the URL, e.g. /student/{id}
`@ModelAttribute`	Bind form fields to an object
`@Autowired`	Dependency injection (Session 4)
`@Entity, @Id, @GeneratedValue`	JPA entity mapping (Session 6)

10.3 Thymeleaf

Syntax	Meaning
`${attr}`	Variable expression: read a model attribute
`*{field}`	Selection expression on th:object
`@{/path}`	Link URL expression
`th:text, th:field, th:object`	Set text, bind a form field, choose form object
`th:if="${cond}", th:each="s : ${list}"`	Condition and loop

10.4 Useful HTTP status codes

Code	Meaning
`200 OK`	Success
`302 Found`	Redirect (e.g. redirect:/dashboard)
`400 Bad Request`	Invalid input / binding error
`403 Forbidden`	Not allowed (Spring Security, missing CSRF token)
`404 Not Found`	No mapping for the URL
`405 Method Not Allowed`	Wrong HTTP method for the mapping
`500 Internal Server Error`	Exception in your code; read the server log

10.5 Official links

- JDK: oracle.com/java/technologies/downloads · adoptium.net
- Java PATH and CLASSPATH: docs.oracle.com/javase/tutorial/essential/environment/paths.html
- Apache NetBeans: netbeans.apache.org/download · Eclipse: eclipse.org/downloads
- Apache Tomcat: tomcat.apache.org · Spring Initializr: start.spring.io
- Maven Repository (search dependencies): mvnrepository.com
- Bootstrap: getbootstrap.com · Font Awesome: fontawesome.com · Thymeleaf: thymeleaf.org